Does Call-Graph Fan-Out Predict Failure Impact in Microservices? A Preregistered Fault-Injection Study

Abstract

Background. One way to decide which services of a microservice system to harden first is to rank them by their position in the dependency graph. An intuitive version of that ranking treats a service that calls many others as the riskiest. When a service fails, however, the services left waiting are its callers. We tested whether the number of transitive callers of a service tracks how many services degrade when it fails, and whether a fan-out-weighted centrality metric does.

Methods. We preregistered the hypotheses and the full analysis, then ran 180 fault-injection runs on Social Network and Hotel Reservation, two applications from the DeathStarBench suite. Each analysable service received a process kill and an injected network delay, with 5 repetitions in each of 36 cells. Dependency graphs were rebuilt from distributed traces, latency was attributed to each service's own spans, and every degraded service was classified by its graph relation to the faulted one. Rank correlations were tested by permutation, with Benjamini-Hochberg correction inside each test family.

Results. On Social Network under kill faults, caller count and the number of degraded callers correlated at $\rho = 0.785$. After correction, none of the 44 confirmatory tests was significant (adjusted $p = 0.3451$), and the latency replication reached the same verdict. Much of the correlation is definitional, since ten of the eleven services degraded every caller they had. The fan-out metric's correlation was negative in all four architecture-by-fault cells and distinguishable from zero in none. Degradation reached callers and almost never callees: no descendant of the faulted service degraded in 179 of 180 runs. The service the fan-out metric ranked most critical had exactly one degraded caller in each of its ten runs. Hotel Reservation was saturated, and its ties kept the best attainable p-value at 0.0952, above the usual threshold.

Conclusions. In these two benchmark applications a failure's reach followed callers, and the fan-out premise found no support. Our test of the caller count did not survive the correction we committed to, so we present the caller ranking as the better-motivated choice, not as a demonstrated predictor. We also recommend two measurement steps that this study needed:

attributing latency to each service's own spans, and checking whether the outcome simply restates the predictor. Every number reported here regenerates from logged data.

1. Background

A microservice application is built from many small services that call one another over the network (Dragoni et al., 2017). Each service can be changed and deployed on its own, which is much of the appeal. The price is that a fault in one service can show up far from where it started, and industrial experience suggests such faults are slow to diagnose (Zhou et al., 2021). Teams that run these systems therefore want to know, before anything breaks, which services to harden first. Injecting faults into every service to find out is expensive. A cheaper option is to read the answer off the dependency graph.

Graph measures have a long record as indicators of importance and risk. The standard notions of centrality were set out for social networks (Freeman, 1978), and network measures computed on software dependency graphs have been used to predict which components will have defects (Zimmermann & Nagappan, 2008). Applied to a service graph, the intuition most people bring is that a service which calls many others is a hub, and that hubs are dangerous. This project began from that intuition. Its original metric gave extra weight to fan-out, the number of services a service calls directly, on the reasoning that a service with many callees cuts many connections when it fails.

A simple argument points the other way. When a service fails, the services that wait on its answers are the ones that call it. Its callees lose a client and see less traffic. If that argument holds, the number of services that degrade should follow a service's transitive callers, its ancestors in the graph, and a fan-out ranking should go wrong precisely for the services it rates highest. Social Network's composition service shows the conflict in one example. It calls 7 services directly, and the fan-out metric ranks it first of the 11 services we analysed. It has a single caller, and by caller count it ranks last.

We tested the caller-based prediction, and the fan-out metric alongside it, in a preregistered fault-injection campaign (Nosek et al., 2018) on two applications from the DeathStarBench suite (Gan et al., 2019). We formulated the caller-based hypothesis after a pilot study (preregistration, §1.3) and locked it, together with every analysis choice, before any campaign data existed, keeping the

fan-out metric as a comparator. The campaign completed all 180 planned runs. Every analysable service received both fault types, a process kill and an injected network delay, with 5 repetitions per cell. We rebuilt each dependency graph from traces, attributed latency to the spans each service emits itself, and classified every degraded service by its position relative to the faulted one, so that degradation of callers and of callees could be counted apart.

The preregistered hypothesis was not confirmed. On Social Network under kill faults, caller count and the number of degraded callers correlated at $\rho = 0.785$, yet none of the 44 tests in the confirmatory family survived correction, and the replication under latency faults gave the same answer. Much of that correlation is definitional, because the outcome counts a subset of the predictor and most services degraded every caller they had. The fan-out metric did no better. Its correlation with degraded callers was negative in every architecture and fault type and was never distinguishable from zero. The plainest result needs no test at all: degradation reached callers and almost never callees, with no descendant of the faulted service degraded in 179 of 180 runs.

This paper makes four contributions. First, it reports a preregistered test of whether caller count, or a fan-out-weighted centrality metric, tracks measured failure impact in two benchmark applications, with its null result and with the power ceiling the design imposed. Second, it documents a measurement pitfall, in which a request's end-to-end latency is charged to every service on its path, and its effect in the pilot, where it flagged services unrelated to the faulted one in the graph. Third, it reports an observation on the direction of propagation that held in both applications and under both fault types. Finally, it provides an artifact in which every number in the paper is produced by a script from logged campaign data.

An earlier analysis from this project was withdrawn after a self-audit found results that had not come from measurement; §7 describes what happened and the safeguards that followed.

The paper is organised as follows. §2 reviews related work, and §3 describes how the faults were injected and how degradation was measured. §4 reports the results, which §5 discusses. §6 sets out the threats to validity, §7 records the audit together with every deviation from the preregistration, and §8 concludes.

2. Related Work

2.1 Graph measures as risk indicators

Centrality has well-established definitions. Freeman's treatment separated several distinct ideas of what makes a node central, some based on a node's direct ties and some on the shortest paths that pass through it, and gave them normalised forms that are still in use (Freeman, 1978). Path-based measures were expensive to compute on large graphs until Brandes published a fast algorithm for betweenness (Brandes, 2001). NetworkX 3.6.1 (https://networkx.org), the graph library we use (Hagberg, Schult & Swart, 2008), implements that algorithm.

Software engineering adopted these measures early. Zimmermann and Nagappan computed network measures on the dependency graph of a software system and found that they helped predict which components would contain defects (Zimmermann & Nagappan, 2008). That work and ours share a premise, that a component's position in a dependency graph says something about its risk. The outcomes differ. A defect count describes where bugs accumulate over a release, while we ask how far a run-time failure spreads once it has happened, which is a property of the call pattern under load and not of the code alone. We are not aware of work that tests a centrality ranking of microservices against failure propagation measured in a controlled, preregistered campaign, though related ideas appear in the root-cause literature discussed in §2.4.

2.2 Fault injection and chaos engineering

Deliberately injecting faults into a running system is now a standard way to learn how it behaves under stress. Basiri and colleagues describe the practice as it developed at Netflix, where engineers run controlled experiments on production systems to build confidence that they tolerate failure (Basiri et al., 2016). For microservices specifically, Gremlin offered a framework for systematic resilience testing, injecting faults between services and checking whether the application handles them as intended (Heorhiadi et al., 2016). Lineage-driven fault injection takes a different route. It reasons backwards from how a correct result was produced to choose the faults most likely to expose a bug, so that fewer experiments are wasted (Alvaro, Rosen & Hellerstein, 2015).

Our use of fault injection is narrower than any of these. We do not search for bugs or validate fallbacks. We inject two fault types into each service in turn, in a fixed and balanced design, and

use the counts of degraded services as the outcome of a structural prediction. The faults are a measuring instrument here.

2.3 Call graphs from traces

Distributed tracing makes it possible to recover a service graph from what the system actually does. Dapper established the model most tracing systems still follow: a request carries a trace context across service boundaries, and each unit of work is recorded as a span with a parent, so a request becomes a tree of spans (Sigelman et al., 2010). Reading parent and child spans from different services gives a caller-callee edge, which is how we built our graphs.

Graphs built this way depend on the traffic that produced them. Luo and colleagues analysed call graphs from Alibaba's production traces and found them to be highly dynamic, changing from one request to the next (Luo et al., 2021). Our benchmark graphs are small and our workload is fixed, so we see far less variation, but the same dependence on traffic affects our predictor, as §6.6 explains. The Hotel Reservation graph we observed already differs from the one the benchmark documents.

2.4 Root-cause localisation

A large body of work starts from the other end of the problem. Given an anomaly that is already happening, root-cause localisation tries to find the service responsible. MicroRCA is one example. It builds a graph of the running system and ranks candidate services by how their behaviour relates to the observed performance problem (Wu et al., 2020). An industrial survey by Zhou and colleagues documents how hard such diagnosis is in practice, and contributes a benchmark system with injected faults for studying it (Zhou et al., 2021).

These methods use the graph after the fact, together with live metrics. Our question comes earlier: whether graph structure alone, before any fault occurs, indicates which services will do the most damage when they fail. The two problems share the dependency graph but ask different things of it. The propagation direction we observe in §4.5 may still be of interest to diagnosis work, though we did not study any diagnosis method.

2.5 Preregistration and multiple testing

Preregistration separates the analyses planned before data are seen from those devised afterwards, so that a reader can tell a test of a prediction from a search through the data (Nosek et al., 2018).

We locked every hypothesis and analysis choice before the campaign, logged every later departure, and label each post-hoc analysis as such. With 44 tests in each family, some correction for multiple testing was unavoidable. We control the false discovery rate with the Benjamini-Hochberg procedure (Benjamini & Hochberg, 1995), applied separately within each family, which keeps the expected proportion of false positives among reported discoveries at the chosen level while giving up less power than a family-wise correction would.

3. Methods

3.1 Applications and dependency graphs

All experiments used two applications from DeathStarBench, Social Network and Hotel Reservation (Gan et al., 2019), built from one pinned upstream commit, `6ecb097`. The code under test was therefore the same in every run. The two applications have different shapes. Social Network is deep and fans out from a single composition service, while Hotel Reservation is shallow and close to a star around its gateway. That difference ended up deciding what each application could show at all (§4.1).

We did not take the dependency graph from documentation. A diagram shows the calls a system can make, and our hypothesis concerns the calls it does make under the workload we perturb. We therefore rebuilt each graph from distributed traces collected with Jaeger 1.62.0 (https://www.jaegertracing.io), adding a directed edge from caller to callee wherever a span from one service is the parent of a span from another (Sigelman et al., 2010). The Social Network graph came from 2,449 traces holding 32,847 spans, and the Hotel Reservation graph from 3,446 traces holding 31,155 spans. The graphs have 12 and 8 service nodes, with 17 and 7 observed call edges. Both are frozen as files, and every predictor is computed from those files, so no later change of workload can shift a structural value.

The observed Hotel Reservation graph differs from the documented one. Of the 8 call edges implied by the published service tree, our traces recover 6. Two never appear under this workload: `reservation→user` and `search→profile`. One edge appears that the tree does not list, `frontend→profile`, and we infer that it stands in for the missing `search→profile`, with the gateway fetching profiles directly. The difference matters for one service. `user` has a single observed caller, `frontend`, where the documentation implies two, so its ancestor count is 1.

With the documented graph, `user` would carry a predictor value that no traffic in our experiments could have produced. We report the divergence and keep the observed graph.

3.2 Analysis set and gateway exclusion

Every correlation excludes the one service in each application that has no caller in the graph: `nginx-web-server` in Social Network and `frontend` in Hotel Reservation. A gateway receives traffic only from the load generator, which is not a node in the graph, so its ancestor count is zero by construction and cannot vary. Killing it also removes the only way into the application, which leaves the outcome undefined. The gateways stay in the paper as labelled qualitative examples. Excluding them leaves 11 services in Social Network and 7 in Hotel Reservation. In Hotel Reservation it also removes the service with the highest fan-out, a scope limit we return to in §6.

3.3 Predictors

The primary predictor is `ancestor_count`, the number of services that can reach a given service through the dependency graph, in other words its transitive callers. Eleven further graph measures are kept as comparators: the hybrid criticality metric described below, degree, in-degree, out-degree, betweenness, closeness, eigenvector centrality, PageRank, a composite rank, descendant count and dominator subtree size. The plan forbade dropping any of them once results were seen, and none was dropped. Centralities follow the standard definitions (Freeman, 1978), and betweenness is computed with the algorithm of Brandes as implemented in NetworkX (Brandes, 2001; Hagberg, Schult & Swart, 2008).

The hybrid criticality metric is the measure this project set out to evaluate. It adds three weighted terms: in-degree centrality with weight 0.4, out-degree centrality scaled by a linear fan-out factor with weight 0.4, and directed betweenness with weight 0.2. The weights come from the original proposal. They were picked by an empirical search during the withdrawn earlier analysis, which means they were not fixed in advance of any data, and we kept them unchanged for the campaign. The locked plan called for tuning the weights on Social Network and freezing them for Hotel Reservation, and that step was never run (Deviation 10). Since that earlier search was tuned in the metric's favour, the departure is an unlikely reason for the metric to show no association.

Two predictor-architecture pairs were declared degenerate in advance, because the predictor is constant across the analysis set and Spearman's ρ is then undefined. Eigenvector centrality is 0

for every analysed Social Network service, the reconstructed graph being acyclic, and every Hotel Reservation service has the same in-degree, each having exactly one caller. Both pairs are left out of the test family. A degenerate graph property is not a null result, and entering it as one would dilute the false-discovery-rate correction. That leaves 22 testable pairs of the 24 considered.

3.4 Fault injection protocol

Every run follows the same sequence. We start the application and drive it with Locust 2.46.3 (https://locust.io) at 25 concurrent users. Each simulated user waits between 0.5 and 1.5 seconds between requests, and each request is drawn from a fixed weighted mix of tasks. On Social Network the weights are 3 for composing a post, 5 for reading a home timeline and 2 for reading a user timeline. On Hotel Reservation they are 6 for a hotel search, 2 for a reservation, 2 for a user lookup and 1 for recommendations. After 60 seconds of warm-up, so that caches and connection pools settle, we record a 60-second fault-free baseline, whose length was identical in all 180 runs. The fault is then injected and held for 30 seconds, the recovery probe runs, and a 20-second cooldown separates one run from the next.

Faults are injected with Pumba (https://github.com/alexei-led/pumba) against the target service's container, using the Docker 29.8.0 client (https://www.docker.com). A kill fault sends SIGKILL, which ends the process without letting it drain connections or answer requests in flight. A latency fault applies a `tc netem` (https://wiki.linuxfoundation.org/networking/netem) rule that adds 500 ms of delay with 100 ms of jitter to the container's network interface. The kill asks what happens when a dependency vanishes; the delay asks what happens when it stays up but stops being timely. Because `pumba netem` blocks for its whole declared duration, the runner launches it as a separate process and records the moment of fault removal when that process exits. Running it synchronously would let the rule expire before the measurement window had opened.

The design is fully crossed and balanced. Each analysable service receives each fault type in 5 repetitions, giving 36 cells and 180 runs, of which 110 are Social Network runs and 70 Hotel Reservation runs. A run whose telemetry is incomplete is discarded and repeated in the same cell. It is never topped up elsewhere and never analysed as zeros. The same rule covered the 17 runs invalidated by deviations 1, 2 and 4 (3, 3 and 11 runs; §7.2), which were collected again in their own cells. Measured from the first to the last end-timestamp of the kept runs, the campaign ran

from 2026-09-26 05:02 to 13:35 UTC, 8.5 hours in all, and lost 0 runs to telemetry failure. Earlier records give 9.3 hours because they also count the 3 pre-launch rehearsal runs.

We kept the raw spans of both the baseline and the fault window for every run. Storing them is cheap, and it means any later change to a metric definition can be replayed against the original bytes instead of being re-run. Several analyses in this paper depend on that.

3.5 Per-service latency and error attribution

The latency definition needed more care than any other part of the method, because the obvious implementation is wrong in a way that is easy to miss.

A distributed trace records a whole request path. A tracing backend asked for the traces involving *X* hands back every trace in which *X* appears, and each of those traces has a root span whose duration is the end-to-end latency of the whole request. A percentile taken over those root spans and called "the p95 of *X*" is really the end-to-end p95 of all requests that passed through *X*, and every service on a request path receives the same value. Error rate has the same flaw if a trace counts as failed whenever any span in it carries an error.

Services on a shared path then become indistinguishable, and any threshold applied to them flags them as a group. Figure 1 shows the effect on Hotel Reservation under a latency fault injected into `search`. The figure uses two pilot method-validation runs, which are not among the 180 campaign runs, and its end-to-end values are reconstructed from their persisted spans in `paper/make_figures.py`. Under that scope three services, `geo`, `rate` and `search`, share an identical baseline p95 in both runs, and a multiplicative threshold flags five services that were not faulted. Two of these, `profile` and `reservation`, are neither ancestors nor descendants of `search`. They share a trace with it and nothing more.

We therefore measure each service's latency on the server spans that service emits. On Hotel Reservation, whose spans carry a span-kind tag, client spans are excluded, since a client span recorded by *X* times the callee as *X* sees it and would pass a downstream delay back to *X*. Social Network's spans carry no span-kind tag, so for Social Network all of a service's spans were used. That applied to 2,598 of 2,598 Social Network samples, and about half of the spans used, 49.7%, are client spans. The run records nonetheless label these samples as server spans. A post-hoc replay limited to server spans altered the outcome of 2 of the 110 runs and none of the service-

level medians, and the primary result came out identical (Deviation 11). An error counts against a service only when one of that service's own spans is tagged as an error, and the rate is divided by the number of traces in which the service appears. End-to-end root-span percentiles are kept only for the gateway, where they are the right quantity, and every returned record carries a scope tag that says which definition produced it.

With per-service attribution, the same Hotel Reservation runs flag one non-faulted service instead of five, and that service is an ancestor of the faulted one. The services that drop out do more than fall below the threshold. Their own service time stays flat, and in one case drops slightly, while the end-to-end value previously assigned to them grows more than a hundredfold. Contention for shared resources would be expected to raise a service's own service time, so this pattern points to a measurement artifact.

3.6 Blast radius classification

For each run we find the services degraded during the fault window and classify each by its graph relation to the faulted node. A service counts as degraded when either of two conditions holds on its own spans. The latency condition requires its fault-window p95 to exceed 2.0× its baseline p95 and to rise by at least 1.0 ms. The error condition requires its error rate to rise by at least 0.05 in absolute terms.

The absolute floor became necessary once percentiles were truly per-service. Some services here have baseline p95 values in the hundredths of a millisecond, and at that scale a purely multiplicative rule reacts to scheduler noise. A service whose p95 rises threefold between two consecutive fault-free windows passes a 2× test without any physically meaningful change. Both conditions are evaluated at full floating-point precision, and values are rounded only for display, since rounding to two decimals first would leave a step about as large as the whole signal of the fastest services. The run executor and the recovery probe share a single implementation of the latency rule, which the offline replay tool also calls. The error-rate rule is repeated in three modules with the same value.

Degraded services are split into three counts that are recorded and analysed separately and never added together. `ancestor_affected_count`, the primary outcome, counts degraded transitive callers of the faulted service. `descendant_affected_count` counts degraded transitive callees. `unrelated_affected_count` counts degraded services that are neither, which would

indicate shared-resource contention or synchronous fan-out. A single combined "blast radius" would throw away the direction of propagation, which is exactly what the hypothesis is about.

Which indicator carries signal depends on the fault type, and the pairing was fixed in the plan. Under kill faults the primary indicator is error rate, and Δp95 is not used. In the pilot, across 10 kill runs, the sign of kill-fault Δp95 split by application. It was negative in every Hotel Reservation run, where p95 fell even as most requests errored, and positive in every Social Network run. Our explanation, that Hotel Reservation's services fail requests quickly when a dependency dies while Social Network's clients wait for a timeout, was not tested. Kill-fault Δp95 is never pooled across architectures. Under latency faults the roles reverse: Δp95 is the primary indicator, and error rate carries no signal because nothing has failed. Each fault type is therefore analysed with its own preregistered indicator.

3.7 Recovery time

Recovery time, T_{rec}, is the number of seconds from fault *removal* until Δp95, Δp99, Δerror-rate and the downstream-affected count are all back within 10% of their pre-fault baseline and stay there for a 5-second confirmation period. Timing from removal matters. Timed from injection, T_{rec} would include the whole fault duration and could never fall below it.

The recovery probe polls at a nominal 1-second interval. Measured as the median gap between successive samples within each run, its effective resolution was 1.4 to 2.4 seconds. Each sample is drawn from a window lagged 5 seconds into the past, because tracing backends flush spans in batches and a window ending at the present instant under-counts. A sample needs at least 20 traces before its percentiles can vote for recovery, so a handful of fast requests cannot end a run early. The tolerance is one-sided, and an empty or thin window counts as no signal. No probe window may reach back past the fault instant.

A run that has not recovered within 180 seconds is recorded as right-censored, with the observation horizon stored beside it, and is never filled in with a number. In the final dataset 0 of the 180 runs were censored. Observed recovery times ranged from 9.72 s to 156.27 s, with a median of 15.74 s. The plan stated in advance that if fewer than 10% of observations were censored, the rank test would become the primary inferential result for T_{rec} and survival models would be reported beside it. That rule fired on the data as collected.

3.8 Statistical plan and preregistration

The analysis was specified and locked before the campaign started (Nosek et al., 2018). The locked document fixes the hypotheses and every analytic choice down to the correction procedure and the reporting commitments. It is supplied as supplementary material, and every later change to it is logged with a date and a note of whether campaign data had been analysed at the time (§7).

The hypotheses under test are H1 and H1b. H1 states that `ancestor_affected_count`, the number of degraded transitive callers, is predicted by `ancestor_count`, the number of transitive callers, and not by out-degree or descendant count, nor by the fan-out-corrected hybrid metric. H1b, which is directional, states that the hybrid metric's Spearman ρ against `ancestor_affected_count` is at most zero.

Associations between a graph predictor and a measured outcome are tested with Spearman's ρ at the service level, after taking the median over each service's repetitions. Significance comes from permutation tests. The asymptotic approximation is unreliable at these sample sizes and with the ties these graphs produce. Confidence intervals are bias-corrected and accelerated bootstrap intervals, with a percentile interval used, and disclosed, where the ties leave the bias correction undefined. The analysis was developed and verified on Python 3.13.2 (https://www.python.org), with SciPy 1.14.1 (https://scipy.org) supplying ranks and the reference Spearman implementation, and statsmodels 0.14.6 (https://www.statsmodels.org) the Cox and log-rank models.

Tests fall into three families, each corrected separately with the Benjamini-Hochberg procedure at $q = 0.05$ (Benjamini & Hochberg, 1995). The confirmatory family holds 44 tests on kill faults. The replication family holds 44 tests on latency faults and is reported as a replication. A third, exploratory family covers secondary outcomes and carries no confirmatory weight. The locked plan considered pooling all families into one correction and rejected it, on the grounds that the confirmatory and replication families ask the same question under different fault types and should not dilute each other. For the primary outcome, the replication in fact reuses the same outcome values as the confirmatory family (§4.4).

Two gates run before any statistic is computed. One confirms that every expected service was present in every run. The other checks that each run's fault-free baseline is a plausible steady

state, flagging any baseline that is an outlier against the rest of the campaign by median and median absolute deviation. Both must report zero exceptions before analysis can proceed. All 10 pre-analysis integrity checks passed on the final dataset.

4. Results

All 180 planned runs completed, across 36 balanced cells. No run was lost to telemetry failure, no gateway was faulted, and every recovery time was observed. Tables 1 and 2 list the graph predictors and measured outcomes for each service in Social Network and Hotel Reservation.

4.1 The confirmatory test

The primary hypothesis, H1 (§3.8), replaced the original fan-out hypothesis after the pilot. On Social Network under kill faults, `ancestor_count` and the measured `ancestor_affected_count` correlate at $\rho = 0.785$ across 11 services, with a bias-corrected and accelerated bootstrap interval of [0.000, 1.000]. The uncorrected permutation p-value, from 10,000 Monte-Carlo permutations, is 0.0080. The exact value, 0.006926, is in Table 3 (Deviation 9). After Benjamini-Hochberg correction across the 44 tests of the confirmatory family, the adjusted p-value is 0.3451.

At $q = 0.05$, not one of the 44 confirmatory tests is significant. The primary test has the smallest uncorrected p-value in the family, and its adjusted value is about 6.9 times the threshold. Table 4 gives the primary test in each family and architecture, and Table 5 summarises both families. The preregistered hypothesis is not confirmed. The point estimate is large and positive, the correction we committed to in advance rejects it, and the confidence interval runs from no association to a perfect one.

Two features of the data qualify that estimate further, and they matter more than the p-value.

The first is saturation. If every service's measured blast radius equalled its graph ancestor count, a correlation of 1.0 would follow by construction and would test nothing. On Hotel Reservation that is exactly what happened: each service's measured outcome matched its graph predictor in every run, under both fault types and at both run and service level. Hotel Reservation is therefore tautological at the unit of analysis, and we report every Hotel Reservation result as directional only. It can neither confirm the hypothesis nor disconfirm it.

Social Network is not fully saturated. Its run-level saturation is 0.909 under kill faults, which is why ρ is 0.785 and not 1.0. Figure 2 shows the pattern. Ten of the eleven services lie exactly on the line $y = x$. One departs from it: `user-service`, which has 4 graph ancestors and registers 2 degraded ancestors in all five of its kill repetitions. The empirical content of the Social Network correlation, as distinct from its definitional content, rests on that one service.

We examined this with a leave-one-out recomputation that repeats the correlation eleven times, withholding one service each time (Table 6, Fig. 3). We designed this check after seeing the confirmatory results, so it is post hoc and carries no inferential weight. Across the eleven recomputations ρ ranges from 0.696 to 1.000. Withholding `user-service` gives ρ = 1.000, a change of +0.215 that is 2.4 times the next largest swing, −0.089 for `compose-post-service`. Only one exclusion moves the correlation much, and it is the one that removes the single service that stops the outcome from restating the predictor.

4.2 The hybrid criticality metric

The metric this work set out to evaluate combines inbound centrality with a fan-out-scaled outbound term, on the assumption that a service calling many others matters because its failure breaks many downstream links. The preregistered secondary hypothesis, H1b (§3.8), expected this premise to point the wrong way.

Table 7 gives all four architecture-by-fault cells. They are not independent. The outcome vectors coincide across fault types, so the cells contain 2 distinct comparisons, one per architecture. The correlation between `hybrid_criticality` and `ancestor_affected_count` is negative in every cell: ρ = −0.144 on Social Network under both kill and latency faults, and ρ = −0.258 on Hotel Reservation. After correction, no cell's correlation can be told apart from zero. The adjusted p-values are 0.9925 and 0.9152 on Social Network and 1.0000 on Hotel Reservation, and the Social Network interval runs from −0.810 to 0.698.

So the hybrid metric has no detectable association with measured blast radius, even though every point estimate is negative. The direction preregistered under H1b appears consistently, but its size cannot be separated from zero, and we make no claim of a negative relationship.

4.3 The motivating case

The metric was first motivated by `compose-post-service`, the composition service of Social Network, which calls 7 services directly and has 10 transitive descendants. Because of that fan-out, the hybrid metric ranks it first of the 11 analysed services, with a score of 0.302. By transitive callers it ranks last, having a single graph ancestor. Figure 4 shows the full rank inversion across the eleven services.

Measurement sides with the caller-based ranking. Faulting `compose-post-service` left exactly one degraded ancestor in every run. In each of its 5 kill repetitions the minimum and maximum were both 1, and the latency runs gave the same result. A post-hoc breakdown shows that in all 10 runs that ancestor was `nginx-web-server`, the gateway. The third kill repetition also recorded two degraded descendants, the only such run in the campaign (§4.5). On the preregistered outcome, the service the original metric rates most critical is the lowest in its application. That outcome counts degraded services; it does not measure requests or functionality lost.

This is a worked case and not a general finding. It concerns one service in one application, and the test across all eleven services, which could have generalised it, is the null result reported in §4.1.

4.4 Replication under a second fault type

The replication family repeats all 44 tests on latency faults. As in the confirmatory family, no test is significant after correction. The smallest uncorrected p-value is 0.0076, with an adjusted value of 0.1848.

The agreement between the families comes with a caveat. Under kill and latency faults, the service medians of `ancestor_affected_count` are the same in both architectures, and at run level the match is nearly as close: for 17 of the 18 services, the five per-run counts are the same under both fault types. On the primary outcome, then, the latency family repeats the kill family's correlations on identical data, so the two cannot disagree. The replication adds no independent evidence about the primary hypothesis.

Taken as a finding in its own right, this says that which services degrade when a given service fails hardly depends on how it fails. A process that disappears and a process that slows down

reach the same neighbours. Recovery is a different matter. The correlation between median recovery times under kill and under latency faults shows moderate agreement on Social Network (0.384) and strong agreement on Hotel Reservation (0.857, n = 7). For recovery time the replication remains informative, since there the two fault types give different values (§4.8).

4.5 Direction of propagation

Across the campaign, degradation travelled to callers and not to callees. No descendant of the faulted service degraded in 179 of 180 runs, and the service-level median, which is the unit of analysis, is zero for each service under both fault types. The single exception recorded two degraded descendants, in a Social Network `compose-post-service` kill repetition. Services unrelated to the faulted node behave the same way, with 179 of 180 runs at zero and one exception.

This is the firmest structural result in the study, and it cannot be expressed as a correlation. Neither outcome varies between services, so 88 of the 154 exploratory tests we attempted have no defined value. A constant outcome does not give a weak correlation. It gives none. We report these tests as undefined, with the reason, in the same way as the two degenerate predictors, and do not enter them as zeros.

4.6 Exploratory findings

Of the 66 testable exploratory tests, 2 survive correction within that family. Both concern the size of latency inflation under latency faults on Social Network. Δp95 correlates with `hybrid_criticality` at $\rho = 0.949$ and with plain `degree` at $\rho = 0.953$, both with an adjusted p of 0.0033.

These results are exploratory and carry no confirmatory weight. Their outcome is not the one H1b concerns. How much latency inflates is a different quantity from how many ancestors degrade, and doing well on the first says nothing about the second, so this does not rescue the hybrid metric. Plain degree centrality also does at least as well as the hybrid metric on this outcome, which gives no reason to prefer the fan-out correction over the simpler measure.

4.7 What the design could have detected

Table 8 reproduces the preregistered power analysis against the realised analysis sets. Both sets have the registered size and the registered number of distinct predictor levels, and the ceiling correlations come out exactly as preregistered.

Those ceilings limit what the study could ever show. `ancestor_count` takes only 4 distinct values across the 11 Social Network services and 2 across the 7 Hotel Reservation services, and the ties cap the attainable correlation at 0.941 and 0.791 respectively. For Hotel Reservation the effect is severe. Under the exact permutation test the plan mandates, the best attainable p-value is 0.0952, above the conventional threshold before any correction. The preregistration quoted p = 0.034 for this ceiling, which is the asymptotic value (Deviation 7); the exact value the plan requires is 0.0952. Hotel Reservation could not have reached nominal significance for this predictor whatever outcome we observed. That follows from its tie structure and sample size, not from anything we measured.

4.8 Recovery time

No run was censored, so under the preregistered rule the Spearman rank test is the primary test for recovery time, and the Cox and log-rank models are secondary and uncorrected (Deviation 6). Table 9 gives all three for both predictors. No recovery-time test is significant after correction in either family. The log-rank test is undefined for 10 of the 22 recovery-time tests in each family, including `ancestor_count` on Hotel Reservation, where the predictor has fewer than two distinct terciles.

5. Discussion

5.1 Why fan-out points the wrong way

The direction of propagation is the result we would defend with the least hesitation. In 179 of 180 runs no descendant of the faulted service degraded, and the service-level median of degraded descendants was zero for every service under both fault types. The set of services that degraded also barely depended on how the fault was delivered. For 17 of the 18 services, the five per-run counts were identical whether the service was killed or slowed down.

The reading we find most plausible is that a caller suffers while it waits on a dependency that has died or slowed, whereas a callee simply receives fewer requests and has nothing to wait for. Our measurements fit that reading, but they do not test it. The degradation rule asks only whether a service's own latency or error rate rose past a threshold, and it never observes waiting directly, so the mechanism remains an interpretation.

If the reading is right, a fan-out ranking scores the wrong end of each edge. The composition service in Social Network is the clearest case. It calls 7 services and the fan-out metric ranks it first, yet when it was faulted it had exactly one degraded caller in each of its 10 runs, and in a post-hoc breakdown that caller was always the gateway. Across all services, the metric's correlation with degraded callers was negative in every cell and separable from zero in none (§4.2). We read that as an absence of support for the fan-out premise. The data cannot show a negative relationship.

5.2 A predictor that sits close to its outcome

The caller count correlates with degraded callers partly by construction, since the outcome counts a subset of the predictor. Whenever every ancestor of a faulted service degrades, the two are equal. That happened for every Hotel Reservation service and for ten of the eleven Social Network services.

A post-hoc breakdown of the degraded ancestors makes the point sharper. In all 180 runs the gateway was one of them, and in both applications every analysable service has the gateway among its graph ancestors. Every measured ancestor count therefore includes one unit that is the same for every service. What remains to be explained comes from the intermediate callers, and in Social Network only `user-service` departs from its graph count in a consistent way. It has 4 graph ancestors and registered 2 degraded ancestors in all five kill runs. Its two ancestors that degraded sit on request paths the workload exercises, and its other two showed no degradation in any run. We formed that explanation after seeing the data and have not tested it.

The leave-one-out check, also post hoc, shows what follows from this. With `user-service` withheld, the Social Network correlation becomes 1.000. A reader can fairly treat the Social Network result as resting on one service's worth of independent information. We think the lesson extends past this study in one narrow sense: any structural predictor tested against a structurally

defined outcome needs a saturation measure reported next to the correlation, or the correlation cannot be interpreted.

5.3 What users would see

The preregistered outcome counts degraded services. It says nothing about how many requests fail. To get closer to what a user experiences, we looked after the fact at the gateway's end-to-end error rate under kill faults. This analysis is exploratory and descriptive, and none of its tests enters a preregistered family.

Gateway error followed the request mix. For each faulted service we summed the Locust task weights of the request types whose traces pass through it, and divided by the total weight. The rank correlation between that share and the median gateway error was 0.788 on Social Network and 0.964 on Hotel Reservation. These are descriptive values with no p-value attached. In Hotel Reservation, killing `reservation`, which sits on request types carrying 0.727 of the weight, gave a median gateway error rate of 0.7340. Killing `recommendation`, with a share of 0.091, gave 0.0860. The mix does not explain everything. The Social Network services reached only through the compose request all share a weight of 0.300, and their median error rates still range from 0.1692 to 0.2571.

The hybrid metric's exploratory correlation with gateway error on Social Network was 0.786, not significant after correction within its family (adjusted $p = 0.0913$). Splitting the metric into its weighted terms shows where that association comes from. The in-degree term alone has a rank correlation of 0.715 with gateway error, the fan-out term 0.310 and the betweenness term 0.229. The association the metric shows here follows the count of direct callers more closely than the fan-out correction, so it lends that correction no support.

5.4 Lessons about measurement

Two measurement choices mattered more than we expected when we planned the study. The first is where latency is attributed. Charging each trace's end-to-end latency to every service on the trace makes services on a shared path indistinguishable. In the pilot comparison of §3.5 that shortcut flagged five non-faulted services where per-service attribution flagged one. The correction was in place before the campaign began, so we cannot say what an uncorrected campaign would have concluded. The pilot shows how it would have behaved.

The second is that per-service attribution is only as good as the instrumentation behind it. Social Network's spans carry no span-kind tag, so its per-service latency drew on all of a service's spans, and 49.7% of those were client spans. A post-hoc replay limited to server spans altered 2 of the 110 run outcomes and none of the service-level medians. We were fortunate that the change was this small. A study that depends on span semantics should check them before collecting data, which we did not do.

5.5 Implications for practitioners

For a team that has to rank services before running any experiment, our data favour looking up the graph at callers over looking down it at callees. That recommendation rests on the propagation pattern and on the lack of support for fan-out. The preregistered test of the caller ranking itself did not reach significance, so we do not claim it predicts failure impact. In both applications, the post-hoc breakdown of §5.2 found the gateway in every caller count, and in the exploratory analysis of §5.3 the user-visible error rate tracked how much traffic passed through the failed service. A ranking that weights callers by the traffic they carry may therefore be worth testing, as a new preregistered hypothesis. We did not test it.

The applications are benchmarks, and we did not configure or study containment mechanisms such as retries with back-off or circuit breakers, which could change how far a failure travels. Larger applications would also lift the ceiling that eleven services place on any correlation, and a direct check that injected faults have been removed would close the gap described in §6.5.

6. Threats to Validity

6.1 Sample size and the ceiling on attainable correlation

The unit of analysis is the service, so the sample size is 11 for Social Network and 7 for Hotel Reservation. Those are the sizes the applications have. They were registered in advance, and the realised sets match them exactly. Repetition makes each service's measured outcome more precise, which is why we used 5 repetitions per cell, but it adds no points to a correlation. A study designed this way is bounded by how many services the system has.

Ties tighten the bound. `ancestor_count` takes 4 distinct values on Social Network and 2 on Hotel Reservation, and ties cap Spearman's ρ whatever is measured. The best attainable

correlations are 0.941 and 0.791. The confirmatory null should be read with that in mind. We registered these ceilings before collecting data, so they are a stated property of the design and not an explanation found afterwards.

6.2 Hotel Reservation could not have reached significance

For Hotel Reservation the problem goes beyond low power. Under the exact permutation test the plan mandates, a perfect result with the observed ties, $\rho = 1.0$, yields $p = 0.0952$. That is above the conventional 0.05 threshold before any correction for multiplicity. No outcome we could have measured on this application would have reached nominal significance for this predictor.

The application is also saturated. Each service's measured count matched its graph count in every run, so a correlation computed here only restates how the outcome was built. We report every Hotel Reservation result as directional only, and it should be read neither as replicating the Social Network result nor as failing to. Its value here is as a second, very differently shaped topology on which the propagation-direction result of §4.5 also holds.

6.3 Two applications from one benchmark suite

Both applications come from one benchmark suite, built from one pinned upstream commit. They are research benchmarks. We chose them because they are instrumented and reproducible, which makes the experiment repeatable, but they do not sample the space of microservice architectures in any principled way. We did not configure or study containment mechanisms such as retries with back-off or circuit breakers, which could change how far a failure travels. Nothing here describes production behaviour.

The workload is synthetic as well, fixed at 25 concurrent users with a fixed request mix. Because the dependency graph is rebuilt from observed traffic, a different mix would give a different graph on the same deployment, and both the predictors and the outcomes would move with it (§6.6).

6.4 The primary predictor is close to definitional

The most serious threat to the primary result is conceptual. The outcome counts degraded transitive callers and the predictor counts transitive callers, so the two coincide whenever every ancestor degrades. On Hotel Reservation they always coincide. On Social Network they coincide

for ten of the eleven services, the exception being `user-service`, which registers 2 of its 4 ancestors in every kill repetition.

The whole gap between $\rho = 0.785$ and a correlation of exactly 1.0 therefore rests on one service. The post-hoc leave-one-out check confirms the arithmetic: dropping `user-service` raises ρ to 1.000, a change 2.4 times the size of the next largest. That check was specified after the confirmatory result was known, and we report it as a description of the data.

Before collection we asked whether a predictor this close to its outcome was worth testing at all, and we registered the saturation measure to detect exactly this problem. We still think the test was meaningful, because saturation was not guaranteed in advance and does not fully hold. A reader may nonetheless treat the Social Network correlation as carrying roughly one service's worth of independent information.

This study cannot say why `user-service` departs. In all five of its kill repetitions the two ancestors that degraded both sit on request paths the workload drives, and its other two graph ancestors showed no degradation in any run. That suggests `ancestor_count` is a ceiling, and that how close the outcome comes to it depends on the request mix. We formed this explanation after seeing the data and have not tested it. Testing it would need a workload-weighted predictor and a new preregistration.

6.5 Fault removal is inferred, not verified

For latency faults, the runner infers that the `tc netem` rule has been removed from the fact that the injecting process has exited. That inference failed at least once, and §7 describes the incident and the safeguards added afterwards. The safeguards limit the damage without ruling it out. The cross-run baseline gate catches a rule that outlives the run that created it, because the contamination then shows up in later runs' fault-free baselines. A rule that leaked and then cleared within a single run would go unnoticed, since that run's own baseline comes before its fault.

Latency-fault recovery times therefore carry an assumption we cannot check with the current instrumentation. If a rule persisted into a run's recovery probe, that run's T_{rec} would be inflated and nothing in our pipeline would flag it. Blast-radius counts are less exposed, since they come

from the fault window itself. Closing the gap would need a direct query of the container's queueing discipline, which we did not implement.

6.6 The dependency graph depends on the workload

Rebuilding the graph from traces gives the calls the system actually makes, which we consider the right basis for this question. It also makes the graph a property of the deployment and the workload together. Production call graphs have been found to change from request to request (Luo et al., 2021), and even our fixed benchmark workload shows the effect. Of the 8 edges implied by Hotel Reservation's published architecture, our traces recover 6. Two are never exercised, and one appears that the documentation does not list.

Since `ancestor_count` is computed on the rebuilt graph, a service's predictor value depends on which of its callers the workload happens to drive. Here the `user` service has one observed caller, where the documentation implies two. A different request mix would give it a different predictor value with nothing about the deployed system changed. This limits how far any relationship we report carries over to other traffic, and it is the same mechanism the post-hoc reading of `user-service` in §6.4 relies on, which is one more reason to treat that reading as untested.

6.7 Scope of the measurement

We measure whether a service degrades, by a preregistered threshold on its own latency or error rate. We do not establish why any individual service degrades. We studied two fault types, a process kill and a 500 ms network delay. CPU contention was evaluated in the pilot and dropped before the campaign, because its effect was too small to classify reliably; §7 records that as a scope decision. Other failure modes, from resource exhaustion to slow or corrupted responses, are outside what we measured.

6.8 Social Network recovery times were not replayed

For Social Network, per-service latency included every span a service emitted, client spans among them (§3.5, Deviation 11). The blast-radius outcomes could be replayed on server-only spans from the stored baseline and fault windows. Recovery times could not, because the recovery probe's windows were not stored, so Social Network recovery times stand as measured.

7. Audit and Deviations from Preregistration

7.1 Why this study was rebuilt

An earlier version of this work went to project review. A later self-audit of its pipeline found that some reported results had not come from measurement. Impact values and recovery times presented as experimental findings were random values drawn from ranges selected according to each service's centrality rank, so the metric agreed with them by construction. The metric weights behind the headline correlation had also been chosen by a grid search that maximised that same correlation on the same outcome, with no held-out data. A result produced that way is the best of many tries reported as one test.

We withdrew those results in full. The affected scripts and outputs were moved, not deleted, into a quarantine directory that now holds 13 numbered items, each with a note of what was removed and why. We then rebuilt the pipeline from measurement through to analysis, and everything reported here comes from a new campaign. Two rules follow from that history. Each reported number comes from a script that reads logged data, and any value we cannot trace to a script and an input file is marked as missing. The analysis was also locked before the campaign, and every later change is logged with its date and with whether campaign data had been analysed by then.

One element was carried over. The hybrid metric's weights of 0.4, 0.4 and 0.2 come from the original proposal, where an empirical search had selected them. They were left as they were for the campaign, and the preregistered tuning on Social Network never happened (Deviation 10). That search was set up to favour the metric, so the inherited weights are an unlikely cause of its null result.

7.2 Deviations from the locked specification

Table 10 lists twelve deviations from the locked plan. Together they invalidated 17 runs, each of which was quarantined and collected again in its own cell, which is why the final dataset is complete and balanced at 180 runs. Nine of the twelve invalidated no data. The three that did are deviation 1 (3 runs), deviation 2 (3 runs) and deviation 4 (11 runs). We describe them in the order they occurred, because timing determines how far each could have shaped a result.

One deviation predates all campaign data. It came from the pre-launch rehearsal. The locked rule flagged a service on a multiplicative condition alone, which was safe while per-service

percentiles were still end-to-end values of tens of milliseconds. The per-service definition of §3.5 moved several services into the sub-millisecond range, where a ratio responds to scheduler noise. In the rehearsal, one service's p95 rose threefold between two consecutive fault-free baseline windows and the rule flagged it as degraded. We added the 1.0 ms floor described in §3.6 and discarded the three rehearsal runs.

Two defects appeared during collection. The first was found 79 minutes in and lay in how the runner matched a service name to a container. It matched by substring and took the shortest match. The Hotel Reservation compose project prefixes every container with `hotelreservation-`, which contains `reservation`, so a request to fault `reservation` hit the `geo` container. The run killed `geo` while recording a `reservation` fault, and the restore step then restarted the healthy `reservation`, so `geo` stayed down for the runs that followed. The health check made it worse, since it asked only whether the tracing backend and the gateway answered, and both did without `geo`. We switched to exact matching on the container label, which refuses ambiguous names, required every graph service to be running before a run could start, and collected the three affected runs again. That fix caused the second defect (deviation 3). The new gate compared graph names with compose names, and the two differ for the Social Network gateway, so the campaign aborted on a healthy stack. No data were affected.

The most consequential deviation (deviation 4) was caught by the pre-analysis gates straight after the final run, before any statistic was computed. A network delay rule outlived the process that created it and stayed active through a block of Social Network latency runs until the stack was torn down. The damage had a direction. The contaminated runs' fault-free baselines were inflated by two to three orders of magnitude, and since the rule asks whether fault-window latency exceeds twice the baseline, the faults looked smaller than they were. Services that should have been counted were already slow when the baseline was taken and went uncounted, so the primary outcome was understated by two in most affected runs. No single run record looked wrong; only the comparison across runs exposed it. The eleven runs were collected again. The runner now recreates the target container after every non-kill run, and the baseline-sanity gate that caught the problem is a standing check. §6.5 states the risk that remains.

Four deviations concern the analysis. Deviation 5 records two choices the locked text left open, the level at which the survival models run and which fault types each exploratory outcome

covers; both were settled before the analysis was written. Deviation 6 is a rule firing as written. The plan promoted the rank test to primary for recovery time if fewer than 10% of observations were censored, and none was; the survival models are reported beside it without a separate correction, since they test the same hypotheses. Deviation 7 is an inconsistency in the locked document. Its power section quotes 0.034 as the Hotel Reservation ceiling p-value, which is the asymptotic value the same document forbids for inference, and the exact value is 0.0952 (§6.2). No analysis here used the asymptotic value. Deviation 8 replaced the reference statistical routines with vectorised versions, because the originals were too slow for the number of evaluations required. The change is algebraic, and an equivalence test suite pins it to the reference implementation, including on the tie-heavy inputs this dataset contains.

Deviation 9 was made after the results were known, and we mark it so. The Social Network permutation tests had used Monte Carlo sampling because enumerating all orderings of eleven services looked infeasible. It was not. The null distribution depends only on the two rank vectors, so permuting the vector with more ties enumerates the same distribution from 9,240 distinct arrangements. The exact uncorrected p-value of the primary test is 0.006926, against the locked Monte Carlo estimate of 0.0080, and the adjusted value moves from 0.3451 to 0.3048. A refinement made after a null result looks like an attempt to rescue it, so the safeguards need stating. We applied it to every one of the 43 testable tests, which rules out aiming it at a single result. It changes how the p-value is computed and leaves the test itself untouched. Its direction could not be known in advance, since a resampling with a different seed had landed on the other side of the locked value. The verdict is unchanged, the exact values appear only in a supplementary table (Table 3), and the locked value remains the preregistered one.

The last three deviations were found after the analysis by a read-only check of the repository against its own records. None changed a stored outcome or a locked output. Deviation 10 is the weight protocol described in §7.1; the fixed weights were already in the analysis code before the lock, but the departure was recorded only at this check. Deviation 11 concerns Social Network spans, which carry no span-kind tag, so per-service latency there used all of a service's spans while the run records labelled them server spans (§3.5). Replaying on server spans alone, after the fact, changed the outcome of 2 of 110 runs but no service-level median, and the primary result stayed the same; recovery times could not be replayed (§6.8). Deviation 12 concerns one Hotel Reservation latency run that was attempted twice. The first attempt was interrupted in its

recovery probe when the campaign stopped for deviation 2. It wrote no run record and was never analysed, and the completed second attempt is the run reported.

7.3 Verification

Two checks beyond the preregistered analysis are reported as supplementary. One is the post-hoc leave-one-out recomputation of §4.1. The other is an independent reimplementation of the primary statistic, written without reference to the analysis code and reading only the persisted spans and the dependency graph. It reproduces the reported correlation and agrees with the stored per-run outcomes on which services were affected, not only how many, in all 110 Social Network runs. That shows the analysis code is faithful to its data. Whether the definitions in §3.5 and §3.6 are the right ones is a question no recomputation can settle.

8. Conclusions

We ran a preregistered fault-injection campaign of 180 balanced runs on two benchmark applications to ask whether a service's position in its dependency graph indicates how much damage its failure does. The design crossed every analysable service with two fault types and five repetitions, and the analysis was locked before the first run.

The correlational hypothesis did not survive. The number of transitive callers correlated strongly with the number of degraded callers on Social Network, but after correction not one test survived in either family, the 44 confirmatory and the 44 replication tests alike. The fan-out-weighted metric this project began with showed a negative correlation in every cell and could not be separated from zero in any.

What did hold is structural. Failures reached callers and almost never callees: no descendant degraded in 179 of 180 runs, and the service-level median was zero in every condition. This result needs no inference, and it held in both applications and under both fault types.

For practitioners, this points towards ranking services by their callers and away from ranking them by fan-out, with the caveat that our own test of the caller ranking did not reach significance. For researchers, the study shows two steps that are easy to skip: attributing latency to each service's own spans, and measuring how far the outcome merely restates a structural predictor before reporting a correlation between them. Every number in this paper can be regenerated from the logged campaign data.

Funding

The authors received no funding for this work.

Competing Interests

The authors declare that they have no competing interests.

Data Availability

The code and the raw campaign data, including persisted trace spans and the reconstructed dependency graphs, are available together with the analysis outputs at https://github.com/Daxx133-bot/proj-1-updated and will be archived at Zenodo (DOI: [TO FILL]). The experiments used DeathStarBench at upstream commit 6ecb097. The Locust workloads are defined in `infra/_locustfile_baseline.py` for Social Network and `infra/_locustfile_hotel.py` for Hotel Reservation. Code is released under the MIT licence; data and figures are released under CC BY 4.0. Running `python paper/build_all.py` rebuilds every reported value from the logged data.

AI-Use Statement

An AI assistant, Claude (Anthropic), was used for code development and code review, and for drafting and language editing of the manuscript. The authors ran all experiments. Every number in the paper is generated by a script from the logged campaign data. The authors take full responsibility for the content of the paper.

References

Alvaro P, Rosen J, Hellerstein JM. 2015. Lineage-driven Fault Injection. Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data:331-346. DOI 10.1145/2723372.2723711.

Basiri A, Behnam N, de Rooij R, Hochstein L, Kosewski L, Reynolds J, Rosenthal C. 2016. Chaos Engineering. IEEE Software 33(3):35-41. DOI 10.1109/MS.2016.60.

Benjamini Y, Hochberg Y. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. Journal of the Royal Statistical Society: Series B (Methodological) 57(1):289-300. DOI 10.1111/j.2517-6161.1995.tb02031.x.

Brandes U. 2001. A Faster Algorithm for Betweenness Centrality. The Journal of Mathematical Sociology 25(2):163-177. DOI 10.1080/0022250X.2001.9990249.

Dragoni N, Giallorenzo S, Lluch Lafuente A, Mazzara M, Montesi F, Mustafin R, Safina L. 2017. Microservices: Yesterday, Today, and Tomorrow. Present and Ulterior Software Engineering (Springer):195-216. DOI 10.1007/978-3-319-67425-4_12.

Freeman LC. 1978. Centrality in Social Networks: Conceptual Clarification. Social Networks 1(3):215-239. DOI 10.1016/0378-8733(78)90021-7.

Gan Y, Zhang Y, Cheng D, Shetty A, Rathi P, Katarki N, Bruno A, Hu J, Ritchken B, Jackson B, Hu K, Pancholi M, He Y, Clancy B, Colen C, Wen F, Leung C, Wang S, Zaruvinsky L, Espinosa M, Lin R, Liu Z, Padilla J, Delimitrou C. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19):3-18. DOI 10.1145/3297858.3304013.

Hagberg AA, Schult DA, Swart PJ. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy2008), G. Varoquaux, T. Vaught, J. Millman (Eds.), Pasadena, CA, USA:11-15. Available at http://conference.scipy.org/proceedings/SciPy2008/paper_2/.

Heorhiadi V, Rajagopalan S, Jamjoom H, Reiter MK, Sekar V. 2016. Gremlin: Systematic Resilience Testing of Microservices. 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS):57-66. DOI 10.1109/ICDCS.2016.11.

Luo S, Xu H, Lu C, Ye K, Xu G, Zhang L, Ding Y, He J, Xu C. 2021. Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis. Proceedings of the ACM Symposium on Cloud Computing (SoCC '21):412-426. DOI 10.1145/3472883.3487003.

Nosek BA, Ebersole CR, DeHaven AC, Mellor DT. 2018. The Preregistration Revolution. Proceedings of the National Academy of Sciences 115(11):2600-2606. DOI 10.1073/pnas.1708274114.

Sigelman BH, Barroso LA, Burrows M, Stephenson P, Plakal M, Beaver D, Jaspan S, Shanbhag C. 2010. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report dapper-2010-1. Available at https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/.

Wu L, Tordsson J, Elmroth E, Kao O. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. NOMS 2020 - IEEE/IFIP Network Operations and Management Symposium:1-9. DOI 10.1109/NOMS47738.2020.9110353.

Zhou X, Peng X, Xie T, Sun J, Ji C, Li W, Ding D. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. IEEE Transactions on Software Engineering 47(2):243-260. DOI 10.1109/TSE.2018.2887384.

Zimmermann T, Nagappan N. 2008. Predicting Defects Using Network Analysis on Dependency Graphs. Proceedings of the 30th International Conference on Software Engineering (ICSE '08):531-540. DOI 10.1145/1368088.1368161.